Amrita Vishwa Vidyapeetham
Department of Computer Science and Engineering

Decentralized Compute Marketplace with Verifiable Off-Chain Computation using RISC Zero

Case Study Report — 23CSE335: Blockchain and its Applications

Leela Mahalakshmi (CB.SC.U4CSE23336)
Adwaith A (CB.SC.U4CSE23204)

Repository: https://github.com/Leela0o5/Blockchain_CaseStudy2

Abstract

If you pay someone to run a computation for you, you have two worries. Did they actually do the work properly, and will they get paid fairly once they do? Today a platform in the middle answers both questions, and everyone has to trust it. In this case study we tried to remove that middle layer. We built a small marketplace where a requester locks Ether in a smart contract along with a job (multiplying two matrices), and a worker picks the job up and runs it on their own machine. The worker then hands back a RISC Zero Groth16 proof showing that the right program produced that exact answer. The contract checks the proof itself and pays the worker only if it holds up. We tested the whole flow on a local Hardhat chain using a React dashboard and MetaMask. A genuine proof released the payment, and tampered proofs were turned away without touching the escrow.

1. Why We Picked This Problem

Outsourced computing has a trust gap on both sides. Suppose you give someone a heavy calculation. The only easy way to check their answer is to redo it yourself, which defeats the purpose of outsourcing. The worker has a mirror problem: they spend CPU time and have no guarantee of being paid. Cloud providers paper over this by being a big company everyone already trusts. But that also means one party controls pricing and access, and if it goes down, everything else goes down with it.

A blockchain looked like a good fit for the payment side, since a contract can hold money and release it by rule. The catch is that doing real computation on Ethereum is far too expensive. What we needed was a way to keep the work off-chain and still let the contract check it cheaply. That is exactly what a zero-knowledge virtual machine offers. RISC Zero runs an ordinary Rust program and outputs a *receipt*: a short proof that this specific program, identified by its *image ID*, produced a given output. Checking the receipt on-chain costs a single function call, whatever the size of the original computation.

2. What We Set Out to Build

- A requester can post a job and lock the reward in escrow in the same transaction.
- Workers register once and then claim open jobs.
- The worker computes the result off-chain and produces a Groth16 proof with RISC Zero.
- The contract checks the proof and pays the worker only when it is valid.
- The requester gets a refund if they cancel before anyone accepts the job.
- A web dashboard connected through MetaMask shows all of this live.

To keep the scope realistic we support one deterministic workload, integer matrix multiplication. Uploading arbitrary programs, GPU jobs and large-scale scheduling are outside what we attempted.

3. How the Pieces Fit Together

Figure 1 shows the setup. The orange boxes are the people involved, each with a MetaMask wallet. The red boxes live on-chain. The blue box, the RISC Zero prover, runs on the worker's own computer.

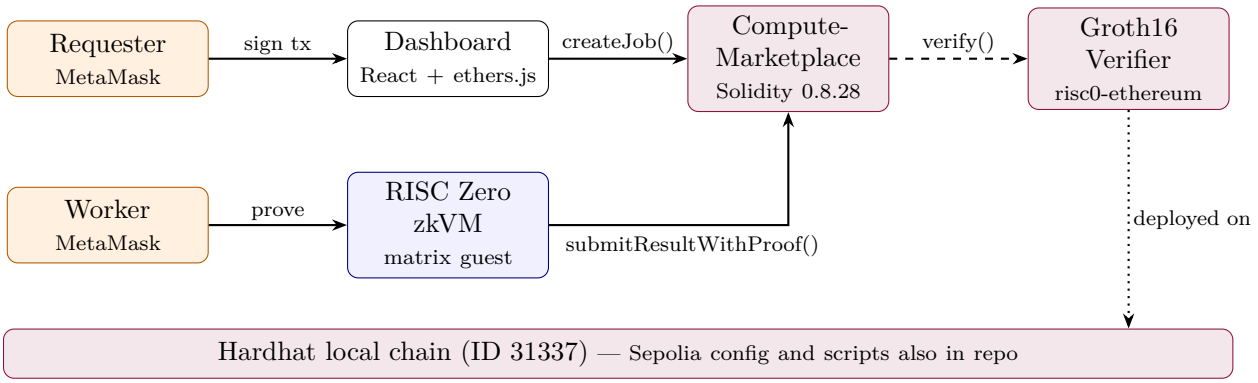


Figure 1: Overall architecture. Red = on-chain, blue = off-chain, orange = users.

The repository splits along that same line. `contracts/` holds the marketplace and the verifier, `risc0/` holds the Rust guest and host, and `frontend/` is the dashboard. We used the official verifier from `risc0-ethereum` without modifying it, because writing our own Groth16 verifier would have been an easy place to introduce a security bug.

The guest program. The guest reads the job ID, a SHA-256 commitment to the input, and the two matrices. It recomputes the commitment itself and stops if it does not match. It then multiplies the matrices using checked arithmetic, so an overflow panics instead of quietly wrapping around. Finally it commits a *journal* containing a version tag, the job ID, the input commitment, and the result matrix. Since the journal carries the job ID and the input hash, a proof made for one job is useless for any other job.

4. The Smart Contract

`ComputeMarketplace.sol` tracks each job through five states: None, Created, Accepted, Completed and Cancelled. It accepts only one guest image ID, which is fixed when the contract is deployed. A requester therefore cannot slip in a different program that would happily prove a wrong answer.

Function	Who calls it	What it does
`registerWorker()`	Worker	Adds the caller to the worker list.
`createJob(imageId, inputData)`	Requester	Payable. Checks the image ID and the input format, then stores the job and holds the ETH.
`acceptJob(jobId)`	Worker	Claims a Created job. A requester cannot accept their own job.
`submitResultWithProof(...)`	Worker	Validates the journal, calls the verifier, marks the job done and pays out.
`cancelJob(jobId)`	Requester	Refunds the reward if nobody has accepted the job yet.
`getJob(jobId)`	Anyone	Read-only. Used by the dashboard.

Table 1: Main contract functions. Owner-only `setVerifier` and `transferOwnership` are not listed.

The payout function is the most important part, so here it is with the bounds-checking helpers left out. Before it even calls the verifier, it checks that the journal belongs to this job, that the input hash matches what the requester stored, and that the result dimensions are right. The status is updated before any Ether moves, and a reentrancy guard sits on top.

```
function submitResultWithProof(uint256 jobId, bytes calldata result, bytes calldata proof)
    external nonReentrant
{
    Job storage job = _jobs[jobId];
    if (job.status != JobStatus.Accepted) revert InvalidJobStatus(jobId, job.status, JobStatus.Accepted);
    if (job.worker != msg.sender) revert OnlyAssignedWorker(jobId, msg.sender, job.worker);

```

```
        _validateJournal(jobId, job.inputData, result); // tag, job ID, input hash, sizes
        IRiscZeroVerifier(verifier).verify(proof, job.imageId, sha256(result)); // reverts if fake

        uint256 rewardAmount = job.reward;
        job.result = result;
        job.proof = proof;
        job.status = JobStatus.Completed; // state first ...
        emit JobCompleted(jobId, job.worker, rewardAmount, result);

        (bool success, ) = job.worker.call{value: rewardAmount}(""); // ... money last
        if (!success) revert PaymentTransferFailed(job.worker, rewardAmount);
    }
```

5. Tools We Used

Tool	How we used it
Remix IDE	We open and compile the Solidity contracts here to walk through the code during the demo.
Hardhat + TypeScript	Local chain (ID 31337), deployment scripts, and the 45 unit tests.
MetaMask	Requester and worker accounts on the "Hardhat Local" network; signs every transaction.
React + Vite + ethers.js	The dashboard: connection status, approved image ID, job table, and a "New job" form.
RISC Zero (Rust)	The matrix guest, plus the host that turns a run into a Groth16 receipt.
risc0-ethereum 1.2.1	The official `RiscZeroGroth16Verifier` contract that the marketplace calls.

6. Running the Demo

1. Start a local node with `npx hardhat node`. Then deploy the verifier and the marketplace, using the matrix guest's image ID.
2. Open the dashboard on `localhost:5173` and connect MetaMask on chain 31337.
3. The requester fills in two matrices and a reward, and confirms `createJob` in MetaMask. The ETH is now held by the contract.
4. A second account registers as a worker and accepts the job.
5. The worker runs the RISC Zero host, which produces the journal and the Groth16 seal.
6. The worker calls `submitResultWithProof`. Once the proof checks out, the reward is transferred and the job shows as Completed.

The script `scripts/verify-real-risc0-receipt.ts` automates the proof part of this flow. It builds a real receipt, sends it through the marketplace, and then deliberately tries a few attacks to make sure they fail.

7. Results

Figure 2 shows the end-to-end proof script. It compiled the guest, generated a genuine Groth16 receipt, and verified it both directly against the verifier and through `ComputeMarketplace`. Job 7 completed and its escrow was released. The last line matters most to us: when the script changed the journal digest, verification rejected it.

```
(base) adwaith@ADWAITH:/mnt/d/Sem7/blockchain/Blockchain_CaseStudy2$ npx hardhat run scripts/verify-real-risc0-receipt.ts
Generating a genuine Groth16 receipt with the Rust host...
matrix_multiply_guest: Starting build for riscv32im-risc0-zkvm-elf
matrix_multiply_guest:     Finished `release` profile [optimized] target(s) in 4.22s


  Real Groth16 receipt verified directly and through ComputeMarketplace.
Verifier address: 0x5FbDB2315678afecb367f032d93F642f64180aa3
Marketplace address: 0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512
Completed marketplace job ID: 7
Image ID: 0x58076cdaca000bc1a82f44c960c49cdf413454e49d1de6913bc1a0332d047932
Journal bytes (hex): 0x525a4d4d5f4a4f55524e414c5f5631000000000000000000000000000000000000000000000000000000000000000078f07b2eaf32cdf5a0689f802346ee98e83f51a1
4d49b5a1f10c46a4f48f6030200000002000000020000001f000000130000005500000037
Journal SHA-256: 0xfeba75155a3a9ab3c799b2767e0fc82002fe00ed743fb2b80869283111103719
Selector-prefixed Ethereum seal: 0xc101b42b2afbfee19019fbfab445f2254ed85363de0b3453612e1538935533481e279aa804b59b017e7ac5065b2a758f1a0058a1b3b8bc2d0f72442bb
8e98082d716557d14e0b32d02c9849cbfc2e2a600937d2e02f5302e1f01ccd8974810777b188b6f17b46e8640cbbc363b3c3a6800dd47692f6d28d13c219874e743d4603c1fd04f2d717d7406cfc
60c16f21b5574008547ea00b601dfe8bb922b3ccbeeff3152f607c59dcdc099aa59553d44d4bd6da8e3931319787df118205621cceae4d2a06e1e1aba128b23072f59f45ebdf1ce9b6da1295b934
66d1e2105ca402109db0acf257e4cb8be4f67eddcea4ae83aa10ad8a9f890af8226890c53cc41990ddb54e7
Altered journal digest was rejected.
```

Figure 2: Output of `verify-real-risc0-receipt.ts`: a real receipt is accepted and an altered journal digest is rejected.

Figure 3 is the dashboard during a demo run. MetaMask is connected to the Hardhat Local network. The approved image ID shown on the card (`0x58076cda...047932`) matches the one in the terminal output. Job #1 appears as Completed, with its 0.01 ETH reward paid from the requester account (`0xf39F...2266`) to the worker (`0x7099...79C8`).

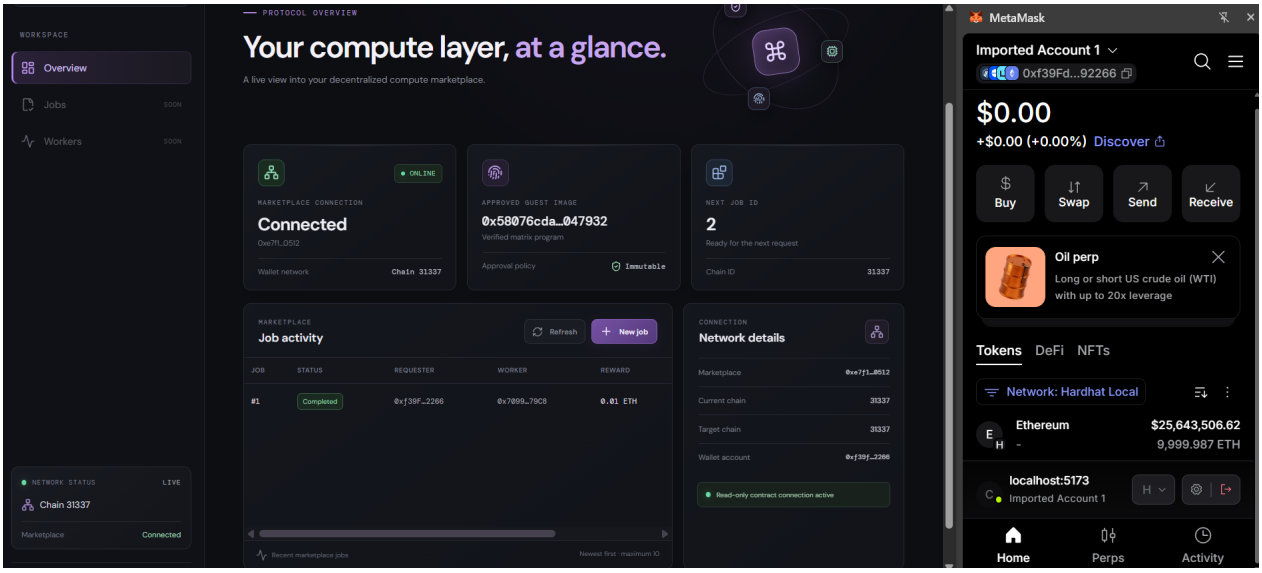


Figure 3: The marketplace dashboard with MetaMask connected and a completed job.

Beyond the happy path, we spent a fair amount of time on the ways this could go wrong:

- **Corrupted seal:** we flipped one byte of the proof. The verifier reverted, the job stayed Accepted, and the escrow balance did not change. Submitting the real proof afterwards still worked.
- **Replay:** we tried to reuse the valid proof for job 7 on a second job, job 8. It was rejected because the job ID is part of the journal.
- **Altered digest:** sent straight to the verifier, it was rejected.
- **Unit tests:** 45 Hardhat tests cover registration, escrow, cancellation, owner controls and every journal check. These run against a mock verifier so they stay fast, and the real-proof script above covers the actual cryptography.

8. What We Learned and What's Next

The biggest lesson was that verifying the proof is not enough on its own. A perfectly valid proof can still belong to the wrong job or the wrong input. Most of our contract code ended up checking

that the journal is tied to *this* job, and only then trusting the verifier. Using RISC Zero also meant writing ordinary Rust instead of hand-building circuits, which is the only reason a two-person team could finish this in a semester.

There is plenty left to do. We would like to deploy and test on Sepolia (the config and scripts are already in the repo), add worker staking so that accepting a job and walking away costs something, and give requesters a timeout refund. Supporting more than one approved program would make the marketplace genuinely useful. Further out, remote proving through Bonsai would mean workers no longer need a powerful machine, and deploying on a Layer-2 network would bring gas costs down.

References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.

[2] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper.

[3] RISC Zero zkVM documentation, https://dev.risczero.com.

[4] risc0-ethereum repository, https://github.com/risc0/risc0-ethereum.

[5] Solidity documentation, https://docs.soliditylang.org.

[6] Project repository, https://github.com/Leela0o5/Blockchain_CaseStudy2.